

Relatório

Aluno: Mateus Crespo

Disciplina: INF01087 – Introdução à Ciência da Computação

Professor: Mateus Grellert

Data de entrega: 13/06/26

Parte 1 – Escolha da Ferramenta:

- Nome da ferramenta: Gemini
- Data de uso: 12/06/2026
- Versão (gratuita ou paga): gratuita
- Acesso à internet: Sim

Parte 2 - Explicação Conceitual:

- **Resposta Prompt simples:**

Recursão em programação é uma técnica onde uma **função chama a si mesma** para resolver um problema.

A ideia central é pegar um problema grande e complexo e dividi-lo em pedaços menores e idênticos. Em vez de usar uma estrutura de repetição tradicional (como `for` ou `while`), a função vai se "autoinvocando" até chegar a um resultado.

Pense na recursão como aquelas bonecas russas (Matrioscas): para chegar na menor boneca possível, você precisa abrir uma boneca maior, que contém outra menor, e assim por diante.

Os Dois Pilares da Recursão

Para que uma função recursiva funcione corretamente e não trave o computador em um loop infinito, ela obrigatoriamente precisa de duas coisas:

1. **Caso Base (Condição de Parada):** É a condição que diz quando a função deve parar de chamar a si mesma. É o menor subproblema possível, que tem uma resposta direta.
2. **Caso Recursivo:** É a parte da função onde ela chama a si mesma, mas sempre modificando o argumento para que ele se aproxime, passo a passo, do caso base.

- **Resposta Prompt elaborado:**

O que é Recursão?

Em termos simples, a recursão acontece quando **uma função chama a si mesma** para resolver um problema.

Imagine que você precisa resolver um problema grande. Em vez de tentar resolver tudo de uma vez com um laço (`while` ou `for`), você divide o problema em uma parte menor e passa essa parte para a própria função resolver. Ela vai se dividindo até chegar em uma resposta óbvia.

A Regra de Ouro da Recursão

Toda função recursiva **obrigatoriamente** precisa de duas coisas:

1. **Caso Base (Condição de Parada):** É a resposta mais simples possível, onde a função para de se chamar. Sem isso, seu programa entra em loop infinito e você ganha um belo *Segmentation Fault* (ou *Stack Overflow*).
2. **Passo Recursivo:** É onde a mágica acontece. A função chama a si mesma, mas sempre passando um subproblema **menor**, aproximando-se do caso base.

O Clássico Exemplo do Fatorial

Lembra do fatorial na matemática? $5! = 5 \times 4 \times 3 \times 2 \times 1$. Repare que $5!$ é a mesma coisa que $5 \times 4!$. E $4!$ é $4 \times 3!$.

Veja como escrevemos isso em C:

```
#include <stdio.h>

int fatorial(int n) {
    // 1. Caso Base: o fatorial de 0 ou 1 é sempre 1
    if (n <= 1) {
        return 1;
    }

    // 2. Passo Recursivo: n * fatorial de (n - 1)
    return n * fatorial(n - 1);
}

int main() {
    int resultado = fatorial(3);
    printf("O fatorial de 3 eh: %d\n", resultado);
    return 0;
}
```

O que acontece na Memória (A Pilha de Execução)

Quando você chama `fatorial(3)`, o C não consegue resolver o retorno imediatamente porque ele precisa do resultado de `fatorial(2)`. Então, o computador "pausa" a execução atual e empilha uma nova chamada.

O fluxo real na memória funciona exatamente assim:

1. `fatorial(3)` entra na pilha → espera pelo `fatorial(2)`
2. `fatorial(2)` entra na pilha → espera pelo `fatorial(1)`
3. `fatorial(1)` entra na pilha → **Cai no Caso Base!** Retorna 1.
4. A pilha começa a desempilhar resolvendo as contas de trás para frente:
 - a. `fatorial(2)` recebe 1 e faz: $2 \times 1 = 2$
 - b. `fatorial(3)` recebe 2 e faz: $3 \times 2 = 6$
5. O programa finalmente retorna 6.

Resumo

A recursão é uma técnica onde uma função chama a si mesma para resolver subproblemas menores de um problema maior. Para funcionar corretamente e evitar loops infinitos, ela exige obrigatoriamente um caso base que interrompa as chamadas e um passo recursivo que aproxime o cálculo dessa parada. Na prática, o sistema operacional gerencia essas chamadas empilhando-as na memória e resolvendo-as de trás para frente assim que o caso base é atingido.

- Comparação das respostas:

Parte 3 - Geração de Código:

- **Prompt utilizado:**

Escreva um programa que leia números digitados pelo usuário e, ao final mostre a média, o maior e o menor valor digitados.

- **Código gerado:**

Inicializa as variáveis

soma = 0 contador = 0 maior = None menor = None

```

print("Digite os números (ou digite 0 para encerrar):")

while True: numero = float(input("Digite um número: "))

# Condição de parada
if numero == 0:
    break

# Somando e contando para a média
soma += numero
contador += 1

# Definindo o maior e o menor valor
if maior == None or numero > maior:
    maior = numero
if menor == None or numero < menor:
    menor = numero

```

Exibição dos resultados (só calcula se pelo menos um número válido foi digitado)

```

if contador > 0: media = soma / contador
print("\n--- Resultados ---")
print(f"Quantidade de números digitados: {contador}")
print(f"Média dos valores: {media:.2f}")
print(f"Maior valor digitado: {maior}")
print(f"Menor valor digitado: {menor}")
else: print("\nNenhum número válido foi digitado.")

```

- **Explicação do código**

- **while True:** Cria um loop infinito que só é interrompido (break) quando o usuário digita 0.
- **maior = None e menor = None:** Começam vazios. No primeiro número que você digita, o programa entende que esse único número é, ao mesmo tempo, o maior e o menor até o momento.
- **f"{media:.2f}":** Formata a média para exibir apenas 2 casas decimais, evitando que apareça um número gigante na tela caso a divisão não seja exata.

- **Teste 1:**

- **Teste 2:**
- **Análise Crítica:**

Parte 4 – Melhorando o Prompt:

- **Prompt utilizado:**

Escreva um programa em C que leia números inteiros digitados pelo usuário, um por vez, terminando quando o usuário digitar -1. Ao final, mostre a média, o maior e o menor valor digitados. Não utilize nenhuma biblioteca além da stdio.h Lembre-se de adicionar comentários em todas etapas.

- **Código gerado:**

```
#include <stdio.h>
```

```
int main() {
    // Declaração das variáveis
    int numero;
    int soma = 0;
    int contador = 0;
    int maior, menor;
    float media;

    printf("Digite números inteiros (ou -1 para encerrar):\n");

    // Loop infinito que será interrompido quando o usuário digitar -1
    while (1) {
        printf("Digite um número: ");
        scanf("%d", &numero);

        // Verifica se o número digitado é a condição de parada
        if (numero == -1) {
            break; // Sai do loop imediatamente
        }

        // Se for o primeiro número digitado, ele será o maior e o menor inicial
        if (contador == 0) {
```

```

        maior = numero;
        menor = numero;
    } else {
        // Atualiza o maior valor se o número atual for maior que o guardado
        if (numero > maior) {
            maior = numero;
        }
        // Atualiza o menor valor se o número atual for menor que o guardado
        if (numero < menor) {
            menor = numero;
        }
    }

    // Atualiza a soma acumulada e a quantidade de números lidos
    soma += numero;
    contador++;
}

// Verifica se o usuário digitou pelo menos um número válido antes de
testar o -1
if (contador > 0) {
    // Calcula a média (o 'float' garante que a divisão não seja apenas
    inteira)
    media = (float)soma / contador;

    // Exibe os resultados na tela
    printf("\n--- Resultados ---\n");
    printf("Quantidade de números digitados: %d\n", contador);
    printf("Média dos valores: %.2f\n", media);
    printf("Maior valor digitado: %d\n", maior);
    printf("Menor valor digitado: %d\n", menor);
} else {
    // Caso o usuário tenha digitado -1 logo de primeira
    printf("\nNenhum número válido foi digitado.\n");
}

return 0;
}

```

- **Controle de Maior/Menor:** A lógica utiliza o contador `== 0` para garantir que o primeiro número válido digitado inicialize as variáveis `maior` e `menor`. Isso evita o erro comum de inicializá-las com `0` (o que

quebraria a lógica caso o usuário só digitasse números negativos, por exemplo).

- **Casting (float):** Na linha do cálculo da média, usamos (float) soma para converter temporariamente a soma inteira em ponto flutuante. Isso garante que o C faça uma divisão decimal exata (ex: $5/2=2.5$ e não 2).
- **Apenas stdio.h:** Toda a entrada (scanf) e saída (printf) de dados foi resolvida utilizando apenas as funções nativas da biblioteca padrão solicitada.

- **Comparação com a versão anterior:**

Parte 5 – Verificação crítica:

- A resposta estava correta?
- 2. Havia informações incompletas ou incorretas?
- 3. Como a resposta foi verificada?
- 4. A LLM demonstrou excesso de confiança?
- 5. Como o prompt poderia ser melhorado?

Uso responsável das LLMs: